asammdf on NixOS

Complete Installation Guide using Distrobox

This guide walks through the complete process of installing and running **asammdf** — a fast Python reader and editor for ASAM MDF / MF4 measurement data files — on a NixOS system using Distrobox and an Ubuntu 24.04 container.

■ Distrobox is the recommended approach on NixOS because asammdf's PySide6/Qt GUI dependencies conflict with NixOS's system Qt libraries when installed natively.

Reading This Guide

- Commands starting with `[ethan@nixos:~]$` run on your NixOS HOST system.
- Commands starting with ■`[ethan@dev ~]$` run INSIDE the distrobox container.
- Never run container-specific commands on your host — your home directory is shared between both environments!

Step 1 — Add Distrobox & Podman to NixOS

Edit your NixOS configuration (usually `/etc/nixos/configuration.nix`) and add distrobox and podman to your system packages:

```
{ config, pkgs, ... }:
{
  # ... your existing configuration ...

  virtualisation.podman = {
    enable = true;
    dockerCompat = true;
  };

  environment.systemPackages = with pkgs; [
    distrobox
    podman
    # ... your other packages ...
  ];
}
```

Rebuild your NixOS system:

```
sudo nixos-rebuild switch
```

Step 2 — Pull the Ubuntu 24.04 Image

Pull the Ubuntu 24.04 container image using Podman before creating the distrobox:

```
podman pull ubuntu:24.04
```

■ Pulling the image first ensures distrobox uses the correct Ubuntu version and avoids any ambiguity.

Step 3 — Create the Distrobox Container

```
distrobox create --name dev --image ubuntu:24.04
```

Verify it was created:

```
distrobox list
```

Step 4 — Enter the Container

```
distrobox enter dev
```

Your prompt will change to show the ■ box icon, confirming you are inside the container:

```
■[ethan@dev ~]$
```

Step 5 — Update Ubuntu & Install Python

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y python3 python3-pip python3-venv python3-full
```

Step 6 — Install asammdf with GUI Support

Install asammdf including all GUI dependencies. The `--break-system-packages` flag is safe here because we are inside an isolated container:

```
pip install asammdf[gui] --break-system-packages
```

Step 7 — Install Required System Libraries

Install all system libraries required for the Qt GUI to work. Run these three groups of commands:

Core X11 and Qt dependencies

```
sudo apt install -y \
    libnss3 \
    libxcb-cursor0 \
    libxcb-xinerama0 \
    libxcb-icccm4 \
    libxcb-image0 \
    libxcb-keysyms1 \
    libxcb-randr0 \
    libxcb-render-util0 \
    libxcb-shape0 \
    libxkbcommon-x11-0 \
    libxcb-xkb1 \
    libxcb-cursor-dev \
    libegl1 \
    libgl1 \
    libdbus-1-3 \
    libfontconfig1
```

Audio and additional X11 libraries

Note: Ubuntu 24.04 uses `libasound2t64` instead of `libasound2`:

```
sudo apt install -y \
    libasound2t64 \
    libxcb-xfixes0 \
    libxcb-sync1 \
    libxcb-util1 \
    libxrender1 \
    libxi6 \
    libsm6 \
    libice6 \
    libxext6
```

XKB and Qt Wayland support

```
sudo apt install -y libxkbfile1
sudo apt install -y qtwayland5 qt6-wayland
```

Step 8 — Configure Environment Variables

This is the most critical step. NixOS automatically injects its own Qt plugin paths into the distrobox environment, which conflict with PySide6's bundled Qt. We need to override them — but **only inside the container**.

■■ Your home directory (~/) is SHARED between your NixOS host and the distrobox container. Never set these Qt variables unconditionally in ~/.bashrc or ~/.bash_profile — it will break your KDE desktop!

The safe solution uses `/run/.containerenv` — a file that only exists inside the distrobox — to guard the variables:

```
cat >> ~/.bashrc << 'EOF'

# Only runs inside distrobox, never on NixOS host
if [ -f /run/.containerenv ]; then
    export PATH="$HOME/.local/bin:$PATH"
    export QT_PLUGIN_PATH="$HOME/.local/lib/python3.12/site-packages/PySide6/Qt/plugins"
    export QT_QPA_PLATFORM_PLUGIN_PATH="$HOME/.local/lib/python3.12/site-packages/PySide6/Qt/plugins/plat
    export QT_QPA_PLATFORM=xcb
fi
EOF
```

Then make distrobox source .bashrc on entry by adding this to .bash_profile (also safely guarded):

```
cat >> ~/.bash_profile << 'EOF'

# Only inside distrobox - source .bashrc on login
if [ -f /run/.containerenv ]; then
    source ~/.bashrc
fi
EOF
```

Step 9 — Test the Installation

Exit and re-enter the container to confirm everything loads automatically:

```
exit
distrobox enter dev
asammdf
```

The asammdf GUI should now launch. ■

> ■ You can verify the container environment variables are correctly set by running: echo $QT_QPA_PLATFORM — it should print 'xcb'.

Day-to-Day Usage

From now on, to launch asammdf from your NixOS host:

```
distrobox enter dev
asammdf
```

To update asammdf in the future:

```
distrobox enter dev
pip install --upgrade asammdf[gui] --break-system-packages
```

Uninstalling Everything

Remove just the container (keeps distrobox)

This deletes the Ubuntu container and everything inside it including asammdf, all apt packages, and all pip packages:

```
distrobox rm -f dev
```

Remove the .bashrc and .bash_profile changes

Open `~/.bashrc` and remove the block we added. It looks like this:

```
# Delete this entire block from ~/.bashrc:
if [ -f /run/.containerenv ]; then
    export PATH="$HOME/.local/bin:$PATH"
    export QT_PLUGIN_PATH="$HOME/.local/lib/python3.12/site-packages/PySide6/Qt/plugins"
    export QT_QPA_PLATFORM_PLUGIN_PATH="$HOME/.local/lib/python3.12/site-packages/PySide6/Qt/plugins/plat
    export QT_QPA_PLATFORM=xcb
fi
```

Then open `~/.bash_profile` and remove the block we added there too:

```
# Delete this entire block from ~/.bash_profile:
if [ -f /run/.containerenv ]; then
    source ~/.bashrc
fi
```

You can edit both files with any text editor, for example:

```
nano ~/.bashrc
nano ~/.bash_profile
```

Remove distrobox from NixOS entirely

First remove the container, then edit `configuration.nix` and remove the distrobox/podman entries, then rebuild:

```
distrobox rm -f dev
# Edit configuration.nix to remove distrobox and podman entries
sudo nixos-rebuild switch
```

■ Because distrobox shares your home directory, there are no leftover files on your host after removing the container. Your ~/Downloads, ~/Documents, etc. are all untouched.

Troubleshooting

asammdf: command not found

The PATH update hasn't taken effect. Source your bashrc manually:

```
source ~/.bashrc
```

Qt platform plugin errors on launch

The Qt environment variables are not set. Verify they are present:

```
echo $QT_QPA_PLATFORM
echo $QT_PLUGIN_PATH
```

If empty, check that the /run/.containerenv guard in ~/.bashrc is correct and re-source it.

KDE taskbar disappeared / slow boot

This happens if the Qt environment variables were set on the host instead of inside the container. Fix it immediately:

```
# On your NixOS HOST (no ■ in prompt):
# Open ~/.bashrc and ~/.bash_profile and remove any lines
# referencing QT_PLUGIN_PATH or QT_QPA_PLATFORM that are
# NOT inside an 'if [ -f /run/.containerenv ]' block.
# Then restart KDE:
kquitapp6 plasmashell && kstart plasmashell
```

Wrong Ubuntu version / image issues

Always pull the image explicitly before creating the container:

```
podman pull ubuntu:24.04
distrobox rm -f dev
distrobox create --name dev --image ubuntu:24.04
```

Why These Steps Are Necessary

NixOS has a unique system design that causes several non-obvious issues when running GUI Python apps:

• **Shared home directory:** Distrobox mounts your real home directory inside the container. Files you create inside the container (like ~/.bashrc changes) also exist on your host.

• **Qt plugin path injection:** NixOS automatically adds all system Qt plugin paths to QT_PLUGIN_PATH in every shell, including inside distrobox. PySide6 ships its own bundled Qt, and mixing the two versions causes 'undefined symbol' crashes.

• **Ubuntu 24.04 package renames:** Some packages have been renamed with a 't64' suffix (e.g. libasound2t64 instead of libasound2) due to the 64-bit time_t transition.

• **Login shell vs interactive shell:** Distrobox enters as a login shell, which sources ~/.bash_profile but not ~/.bashrc directly. We bridge this by having .bash_profile source .bashrc, guarded by the containerenv check.
